When last you saw the view code for projects, you were busy writing JavaScript. In the interest of keeping this example simple, I'm going to move the code back into the ERB. Having to manage the image back down to the JavaScript is just going to be a distraction here.

The relevant view code looks like this in app/views/projects/show.html.erb. (I've added a reference to the task.user.avatar_url.) I've also removed the task-table class from the div surrounding the table, which will prevent the JavaScript from being activated here:

```
<td class="completed">
  <% if task.complete? %>
    <%= task.user.email %>
    <%= task.completed_at.to_s %>
    <img src="<%= task.user.avatar_url %>" />
  <% end %>
</td>
```

If I were being stricter, I'd argue that the avatar_url is only a view-level responsibility—which would imply a decorator, the same way I talked about for the ProjectsController#index action. Let's hold that thought for a possible refactoring.

The test now fails because user.avatar_url doesn't exist. Let's write tests for it.

These are client unit tests, which assume the existence of an adapter. In this testing plan, the adapter is frequently stubbed in the client unit test. You can use this test as a place to design the adapter's interface with the rest of the application. In this case, the logic from the user object's perspective is simple. The user's email is passed to the adapter and is expected to call a method on the adapter that returns the avatar URL.

The test looks like this:

external/01/spec/models/user_spec.rb

```
describe "avatars" do
  let(:user) { create(:user) }
  let(:fake_adapter) { instance_double(AvatarAdapter) }

  it "can get a twitter avatar URL" do
    allow(AvatarAdapter).to receive(:new).with(user).and_return(fake_adapter)
    allow(fake_adapter).to receive(:image_url).and_return("fake_url")
    expect(user.avatar_url).to eq("fake_url")
    expect(fake_adapter).to have_received(:image_url)
    expect(AvatarAdapter).to have_received(:new)
  end
end
```